Reflexion-lite with Execution Traces: Improving Code Generation with WebAssembly interpreter inside the transformer weights

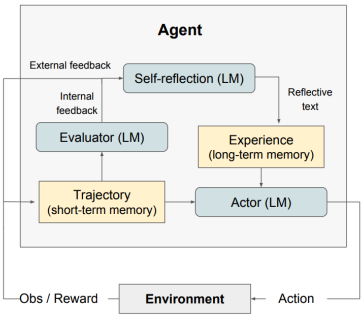
Sergey Egorov
Ivannikov Institute for System Programming of the Russian Academy of Sciences
egorov.sa@phystech.edu

Abstract

We reproduce key ideas from *Reflexion: Language Agents with Verbal Reinforcement Learning* [2] using a lightweight inference-time variant (Reflexion-lite) on a 30-task subset of HumanEval with Mistral-7B. We additionally introduce trace-augmented feedback via the *Percepta Transformer-VM* [1]. Results show consistent gains over baseline and further improvements with execution traces. Code available at `https://github.com/WastingT-me/Reflexion-lite-with-Execution-Traces`.

1 Reflexion Method

Reflexion [2] formulates code generation as an iterative improvement process driven by verbal reinforcement. The system consists of Actor M_a, Evaluator M_e, and Self-Reflection model M_{sr}, with memory mem storing reflections. At step t, the agent produces trajectory τ_t and code c_t, receives feedback f_t, and generates reflection sr_t, after which $c_{t+1} \sim p(\cdot \mid \text{task}, mem, f_t, sr_t)$.



Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection: M_a, M_e, M_{sr}
Initialize policy $\pi_\theta(a_i|s_i)$, $\theta = \{M_a, mem\}$
Generate initial trajectory using π_θ
Evaluate τ_0 using M_e
Generate initial self-reflection sr_0 using M_{sr}
Set $mem \leftarrow [sr_0]$
Set $t = 0$
while M_e not pass or $t <$ max trials **do**
  Generate $\tau_t = [a_0, o_0, \ldots a_i, o_i]$ using π_θ
  Evaluate τ_t using M_e
  Generate self-reflection sr_t using M_{sr}
  Append sr_t to mem
  Increment t
end while
return

Figure 1: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm [2]

Reflections are short natural language explanations identifying failure causes (e.g., boundary conditions, incorrect logic). Crucially, memory accumulates across attempts, enabling improvement without gradient updates but guided by RL-trained policy.

2 Reflexion-lite (R-l)

We remove RL and persistent memory, keeping only in-context iteration: $c_{t+1} \sim p(\cdot \mid \text{task}, c_t, f_t, r_t)$. Loop (max $K = 3$): generate $\rightarrow$ execute $\rightarrow$ feedback (error, traceback) $\rightarrow$ reflection $\rightarrow$ regenerate. **Justification of reproduction:** Due to limited compute and time, we isolate the core mechanism—verbal self-improvement conditioned on execution feedback. Reflexion-lite preserves this mechanism, allowing meaningful reproduction of Reflexion behavior without training. RL + memory will be added in the next version after the deadline.

Preprint.

3 Reflexion+Trace (R+T)

We extend feedback using execution traces from the Percepta Transformer-VM [1], i.e., $f_t = \{\text{error}, \text{traceback}, \text{trace}\}$. The interpreter executes WebAssembly programs inside a transformer; weights are analytically constructed (no training), making execution deterministic. **Attention as addressing.** Attention implements memory addressing analogous to an interpreter with input sequence [program_prefix (WASM) | input | history], where queries retrieve instruction pointer, instruction, stack, and variables. **2D attention for exact lookup.** Keys encode indices as $k_j = (2j, -j^2)$ and queries as $q = (i, 1)$, giving $\arg\max_j k_j \cdot q = i$, which yields exact index selection.

Pipeline Mistral Python → (GPT-5 manual) C → WASM → Transformer-VM → trace → (GPT-5 manual) compression → feedback.

4 Experimental Setup

We evaluate on a 30-task subset of HumanEval using Mistral-7B and Pass@1. The subset is divided into three groups of 10 tasks each: *easy* (solved in one pass), *medium* (requiring multi-step reasoning), and *bug_prone* (boundary conditions or subtle logical errors). For the Reflexion+Trace experiment, due to manual processing constraints, we evaluate on 5 selected tasks (see Table 5). Notably, in *intersperse*, Mistral fails to handle the empty list case; adding execution trace resolves this issue.

5 Results

Table 1: 30 HumanEval tasks Pass@1

Group	Baseline	R-1
Easy	0.5	**0.7**
Medium	0.6	**0.7**
Bug-prone	0.2	**0.6**

Table 2: Trace execution on 5 selected tasks

Task	Baseline	R-1	R+T
below_zero	✓	✓	✓
rolling_max	✓	✓	✓
sum_product	×	✓	✓
intersperse	×	×	✓
has_close_elements	✓	✓	✓

6 Conclusion

We reproduce the central inference-time idea behind Reflexion [2]: iterative improvement through natural-language self-feedback grounded in execution outcomes. Even without RL and persistent memory, Reflexion-lite improves over the baseline on our HumanEval subset. Adding compressed execution traces from Transformer-VM [1] allows the LLM to peek into the interpreter's computation graph, which improves the output again. (even considering the primitiveness of the current pipeline - Percepta doesn't have a Python-to-WASM translation module yet, and the raw trace is too low-level).

Reproducing the article proved to be difficult due to (i) the use of the OpenAI API in the original, which had to be replaced with the much weaker Mistral; (ii) the training phase also proved to be challenging, and RL + memory had to be eliminated; (iii) in practice, the most critical factor was not the iterative scheme itself, but the informativeness of the feedback: sometimes the model received a direct hint (for example, for the error `name 'reduce' is not defined`, the feedback was `from functools import reduce`); (iv) the novelty of the Transformer-VM was also implemented on the fly due to the strict stylization of the readable C code and the lack of a Python translator; (v) the main value of the execution trace lies in its synergy with the original code, which is completely compromised by the manual translation from Python to C and the compression of the trace - in this regard, GPT explicitly specified an empty list in the trace as an exception that breaks the solution for *intersperse* task.

7 Future Work

In the near future, we plan to expand the reproduction to the entire HumanEval dataset and add a full Reflexion (RL + persistent memory). It is also possible to automate the translation of Python to C and the compression of traces using OpenAI API, for example.

References

[1] Percepta AI. Can llms be computers? executing programs inside transformers with exponentially faster inference. https://www.percepta.ai/blog/can-llms-be-computers, 2026.

[2] Noah Shinn, Beck Labash, Ashwin Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.